

CSC343HY Practical 8

Database Security & Indexing

So far you have written queries assuming the database simply answers them. This practical looks at two very different concerns that sit underneath that assumption. First, **security**: a query built carelessly out of user input can be hijacked to do things you never intended, via **SQL injection**. Second, **performance**: once a table is large, scanning every row for every query is unacceptable, so databases use **indexes**, ordered auxiliary structures such as B-Trees, B+ Trees, and hash tables, to jump straight to the data you need.

1. SQL Injection Attacks

The basic idea

SQL injection happens when a program builds a query by directly splicing in user-supplied text, rather than treating that text as inert data. An attacker who controls an input field or URL parameter can insert characters that change the *structure* of the query, not just its values.

TL;DR. SQL injection tries to insert strings that mess with your database if the program isn't prepared for it. The attacker's "password" is not a password at all, it is a fragment of SQL.

Left unguarded, this can lead to spoofed identities, tampered or leaked data, denial of service, or full escalation of privileges, all without the attacker ever needing a valid account.

A vulnerable login query

Consider a login page with two fields, **username** and **password**. A poorly written backend may translate these directly into one line of SQL:

```
SELECT * FROM users
WHERE username = '<your_username>' AND password = '<your_password>';
```

With ordinary input this behaves exactly as intended:

```
SELECT * FROM users WHERE username='tom' AND password='
TheBestSpiderMan123';
```

Breaking it, one step at a time

Attacker enters	Resulting query fragment	Effect
password: '	password=''';	Query becomes invalid SQL; confirms the field is unescaped.
password: ' or '1'='1'	password='' or '1'='1';	The OR clause is always true, so the password check is skipped entirely.
username and password: ' or '1'='1'	username='' or '1'='1' and password='' or '1'='1';	Both checks are bypassed; the query effectively becomes <code>SELECT * FROM users</code> .

*Once both the username and password checks evaluate to **true** regardless of the stored data, the attacker can log in as any account, including an administrator, without ever knowing a real password.*

2. Defending Against SQL Injection

The root cause in Section 1 is always the same: user input was concatenated directly into a SQL string, letting the attacker's characters change the query's structure. Every defense below works by breaking that assumption.

Technique	Idea
Parameterized queries / prepared statements	Send the query text and the user values <i>separately</i> . The database compiles the query structure first, so any quotes or keywords in the values are treated as plain data, never as SQL.
Input validation	Reject or sanitize input that doesn't match the expected shape (e.g. a username with no quotes or semicolons). A defense-in-depth measure, not a substitute for parameterization.
Least privilege	The database account a web app connects as should not be able to DROP tables or read unrelated data, so a successful injection still does limited damage.
Escaping special characters	Escape characters such as ' before inserting them into a query. Fragile compared to parameterized queries, and easy to get wrong.

The fix for the login query. Instead of building a string, use a parameterized query such as `SELECT * FROM users WHERE username = ? AND password = ?;` and pass the user's input as bound parameters. No matter what the attacker types, it can only ever be compared *as a value*, it can never rewrite the query.

3. Indexes

Why bother?

An index is an auxiliary data structure that lets the database retrieve rows without scanning every row in a table for every query.

Pro	Searches (and range queries, for tree indexes) become much faster than a full scan.
Con	Every insert, update, or delete must also update the index, so writes get slower and the index itself takes up space.

There are two broad families covered here: **tree-based** indexes (B-Trees, B+ Trees) and **hash-based** indexes (hash tables).

Indexed Sequential Access Method (ISAM)

ISAM is an older, *static* tree structure: data entries live in leaf pages, with overflow pages chained off a leaf when it fills up. Only the overflow chains change after the structure is built; the non-leaf levels stay fixed. This rigidity is exactly what B+ Trees improve on by allowing the whole structure to grow and shrink dynamically.

4. B-Trees, B+ Trees, and Hash Tables

B-Trees

A B-Tree is a self-balancing tree that keeps data sorted and supports search, insertion, deletion, and sequential access. Both internal nodes and leaf nodes may store key-value pairs.

B+ Trees

A B+ Tree is the structure actually used by most database indexes today. It differs from a plain B-Tree in a few important ways.

	B-Tree	B+ Tree
Property		
Where values live	Both internal and leaf nodes	Only leaf nodes; internal nodes store keys only, as signposts
Sequential access	Not supported	Leaf nodes are linked in a chain
Range queries	Supported, less efficiently	Efficient, thanks to the linked leaves
Typical use	In-memory structures	Disk-based indexes, minimizing disk accesses

Fill-factor rule (order $n = 4$, as used in this practical). The root is exempt. Every other internal (index) node uses at least $\lceil n/2 \rceil$ pointers. Every leaf node holds at least $\lceil (n-1)/2 \rceil$ keys. This is what forces a node to split once it would exceed capacity, and to merge or redistribute once a deletion would drop it below the minimum.

Hash-based indexes

A hash index keeps a collection of buckets $0, \dots, N - 1$, each bucket being one primary page plus zero or more overflow pages. To find a key, apply a hash function h (e.g. $h(x) = x \bmod N$) to locate its bucket, then search only that bucket.

Hash indexes are excellent for *equality* lookups (`WHERE id = 7`) but give no help at all for *range* queries (`WHERE id BETWEEN 3 AND 9`), since keys are scattered across buckets with no order.

Choosing between them

Operation	B-Tree	Hash Table	Notes
Insert	$O(\log n)$	$O(1)$	Hash insertion can degrade with collisions.
Delete	$O(\log n)$	$O(1)$	Same caveat as insertion.
Search (exact)	$O(\log n)$	$O(1)$	Hash worst case is $O(n)$ with a poor hash function.
Find min/max	$O(\log n)$	$O(n)$	B-Trees keep items ordered.
Ordered traversal	$O(n)$	N/A	Hash tables have no notion of order.
Rehash / rebalance	N/A	$O(n)$	B-Trees stay balanced automatically.

A rough rule of thumb: pick a **hash table** for stable (“cold”) data or small, mostly-static datasets queried by exact match; pick a **B-Tree** when the data grows or shrinks quickly, or when you need ordering and range queries.

5. Worked Example: Building a B+ Tree

Throughout, our B+ Tree has order $n = 4$: each node holds up to 4 pointers and 3 keys, so a leaf may hold at most 3 keys and an internal node at most 3 keys / 4 children.

Part I: Inserting 1, 3, 5, 7, 9, 2, 4

Steps 1–4 (inserting 1, 3, 5, 7). Each fits in the single leaf until a fifth key is added:

Leaf: 1* 3* 5* 7*

Step 5 (inserting 9). The leaf would hold 4 keys, over the limit of 3, so it **splits**. Copy the first $\lceil n/2 \rceil = 2$ keys into the left leaf, the rest into a new right leaf, and copy up the middle key (5) into a new root:

Root: 5

Leaves: 1* 3* 5* 7* 9*

Steps 6–7 (inserting 2, then 4). 2 fits in the left leaf. Inserting 4 would overflow the left leaf again, so it splits ($1, 2, 3, 4 \rightarrow 1, 2$ and $3, 4$), pushing key 3 up into the root:

Root: 3 5

Leaves:

1*	2*
----	----

3*	4*
----	----

5*	7*	9*
----	----	----

Insertion algorithm, in one line. Find the correct leaf; insert the entry; if the leaf now has too many entries, split it in half, copy up the middle key to the parent, and repeat one level higher if the parent also overflows. A root split is the only way the tree grows taller.

Part II: Deleting from a full tree

Suppose insertion continued until the tree reached

Root:

3	5	7
---	---	---

Leaves:

1*	2*
----	----

3*	4*
----	----

5*	6*
----	----

7*	8*	9*
----	----	----

Deleting 7. Removing 7* leaves the fourth leaf with {8,9}, which is still at least $\lceil (n-1)/2 \rceil = 1$ key, so no rebalancing of that leaf is needed. But the key 7 that guided searches into that leaf must also be removed from the root:

Root:

3	5
---	---

Leaves:

1*	2*
----	----

3*	4*
----	----

5*	6*
----	----

8*	9*
----	----

Deletion algorithm, in one line. Find the leaf and remove the entry; if the leaf is still at least half full, done. Otherwise, first try to redistribute from a sibling with entries to spare; if that fails, merge the leaf with a sibling, and delete the corresponding key from the parent, a step that can itself cascade all the way up to (and shrink) the root.

6. Exercises

Show your reasoning, not just the final answer.

1. A web form builds this query directly from user input:

```
SELECT * FROM comments WHERE post_id = '<user_input>';
```

- (a) Give a value for <user_input> that would return *every* row in `comments`, regardless of `post_id`.
 - (b) Give a value for <user_input> that would delete the entire `comments` table (assume the driver allows multiple statements per call).
 - (c) Rewrite the query as a parameterized query, and explain in one sentence why your two attacks above no longer work against it.
2. For each scenario, say whether a **B-Tree** or a **hash table** index is the better choice, and justify it in one or two sentences.
 - (a) A table of flight prices that customers frequently query with “show me all flights between \$200 and \$400.”
 - (b) A table mapping session tokens (random strings) to logged-in users, always looked up by exact token.

- (c) An products table for a rapidly growing e-commerce startup, indexed on product ID, with thousands of new products added weekly.
3. A B+ Tree of order $n = 4$ (up to 3 keys per node) currently has a single leaf containing $\{10, 20, 30\}$ and no separate root.
- (a) Insert 25. Show the split: which keys go left, which go right, and which key is copied up.
- (b) Starting again from $\{10, 20, 30\}$ with no root, insert 25, then 5. Draw the resulting two-level tree.
4. Using the tree you built in Exercise 3(b), delete 30. Does the affected leaf need to redistribute or merge with a sibling, or is it still at least half full on its own? Show the resulting tree.
5. A hash index uses $h(x) = x \bmod 4$ with 4 buckets. Insert keys 8, 5, 13, 2, 17 in that order, and show the final contents of each bucket. Which two keys collide, and in what order do they end up in their shared bucket?
6. In one or two sentences, explain why a hash index is a poor choice for a query like `WHERE price BETWEEN 10 AND 50`, even though it would be an excellent choice for `WHERE id = 50`.

7. Common Pitfalls

1. **Building queries by string concatenation.** Any place user input is spliced directly into SQL text is a potential injection point, not just login forms.
2. **Treating escaping as sufficient.** Manual escaping is fragile and easy to miss a case; parameterized queries fix the problem structurally instead of patching symptoms.
3. **Assuming indexes are free.** Every index speeds up reads but slows down writes and costs storage; indexing every column “just in case” is rarely a good trade-off.
4. **Confusing B-Trees and B+ Trees.** Remember: only B+ Tree leaves are linked together, which is exactly what makes range queries and sequential access efficient.
5. **Reaching for a hash index for range queries.** Hash indexes scatter keys with no order; they cannot help with `<`, `>`, or `BETWEEN`.
6. **Forgetting that deletions can cascade.** A leaf merge in a B+ Tree can propagate all the way to the root and shrink the tree’s height, just as a leaf split can grow it.

Before You Leave

Checklist

1. I can explain, with an example, how an unparameterized query lets an attacker bypass a login check.
2. I can describe at least two concrete defenses against SQL injection, and why parameterized queries are the strongest of them.

3. I can explain what an index is for, and the trade-off (faster reads, slower writes) it introduces.
4. I can compare B-Trees, B+ Trees, and hash tables, and pick the right one for a given access pattern.
5. I can insert into and delete from a B+ Tree by hand, including splits, merges, and redistribution.